

Gebze Technical University

CSE 341 — Concepts of Programming Languages · Spring 2026

Project Part 2 — AI Usage Journal (D4) ScoreScript

Student Number: **230104004090**

Name: **FATİH EMRE OĞAN**

Submission Date: **22 May 2026**

This journal documents my substantive interactions with AI tools during the development of ScoreScript for CSE 341 Parts 1 and 2. Each entry records the prompt, the response, what I accepted or rejected, and my critical evaluation. Part 1 entries are dated 2–8 May 2026. Part 2 entries are dated 18–22 May 2026.

Part 1 Entries :

Entry #1	Date: 2026-05-03 Phase: Design AI Tool: Claude Sonnet 4.6
Goal	Choose a suitable domain for my DSL project.
Prompt	I have a CSE 341 Programming Languages course project. I need to design a small DSL. I'm thinking about something related to student grading. What domain would you suggest and why?
Response	AI suggested a DSL called GradeLang where you define courses, grading components like midterm and final, and rules for passing or failing. It used course and rule blocks as the main domain-specific constructs.
Accepted	The general direction of a grade management DSL was good — domain is appropriate for the project and different from handout examples.
Rejected / Modified	The course component, rule, flag structure was too complex. Those nested blocks would make the parser really hard to write. I wanted something simpler where the student is the main thing, not the course.
Errors Caught	AI gave a design that looks more like a config file than a real programming language. The handout shows a different style and AI missed that. It also ignored how hard it would be to implement.
Reflection	<i>AI's first suggestions are usually too complex. Simpler designs are easier to implement and test.</i>

Entry #2	Date: 2026-05-03 Phase: Design AI Tool: Claude Sonnet 4.6
Goal	Add user-defined functions and variable declarations to the grammar.
Prompt	The declaration section is missing two things. I also want user defined functions like: <code>func letterGrade(avg: float) -> string { if avg >= 90 { return "AA" } else if avg >= 85 { return "BA" } else { return "FF" } }</code> And variable declarations like <code>int x = 5</code> or <code>float threshold = 60.0</code> . Both should be in the declaration section before statements. Update the grammar according to what I said.
Response	AI updated the grammar with <code>func_decl</code> and <code>var_decl</code> in the declaration section. It also added a proper expression grammar with arithmetic operators and function call expressions.
Accepted	The general <code>func_decl</code> and <code>var_decl</code> structure. The expression grammar with proper layers for arithmetic was also a good addition.
Rejected / Modified	AI used a separate <code>func_body</code> with <code>func_stmt</code> rule. I later merged this with the general <code>stmt</code> rule to avoid duplication. Also operation syntax was still wrong: "average IDENT IDENT" and "rank IDENT by average". I simplified both later.
Errors Caught	AI forgot to add <code>void</code> as a return type even though functions that only print and don't return a value are needed. Also the expression grammar was missing boolean operators: <code>no &&, , or !</code> anywhere.
Reflection	<i>AI remembered our previous conversation but missed some things I did not explicitly ask for. I had to ask for void and boolean operators separately in later prompts.</i>

Entry #3	Date: 2026-05-03 Phase: Design — E1 AI Tool: Claude Sonnet 4.6
Goal	Ask AI to generate an EBNF grammar from a short description of ScoreScript and compare it with my own design.
Prompt	I want to keep it simpler. The course structure feels too complex for a first DSL. I want something where the student is the main entity. So I define students with their grades, group them and then do things like compute averages, apply a curve, show a transcript, rank them. Something like: student Fatih { math: 85, science: 90 } group ClassZ23 = [Fatih] curve ClassZ23 by 5 transcript Fatih. Can you suggest a design like this with a short example?
Response	AI produced a rough EBNF grammar for a language it called RosterLang. The grammar had student declarations, group declarations, operations and if statements. It also suggested three primitive types: int, bool, string.
Accepted	The general structure of declarations and operations was a good starting point. The observation was correct — every statement starts with a unique keyword.
Rejected / Modified	Several things were wrong compared to my final design. AI wrote average IDENT IDENT and rank IDENT by average. I simplified both to just average IDENT and rank IDENT. Student had no id, firstName, lastName fields. No float type was included. Comment syntax used # instead of //. There was no expression grammar, only arithmetic was mentioned, no boolean operators or precedence layers. The <program> rule mixed declarations and statements instead of separating them.
Errors Caught	Operator precedence was completely missing. AI gave no precedence layers at all. My final grammar has 7 layers from unary to . AI also did not include float type even though averages naturally return decimal values. The # comment syntax conflicted with our Java implementation convention.
Reflection	<i>AI's first EBNF is a useful starting point but misses many details. The biggest gap was operator precedence. AI produced a flat expression grammar with no layering. I had to ask for boolean operators and proper precedence separately in later prompts.</i>

Entry #4	Date: 2026-05-03 Phase: Design AI Tool: Claude Sonnet 4.6
Goal	Ask AI to write the complete final grammar and verify it matches my design decisions.
Prompt	Write the complete final grammar one more time with all the changes we made.
Response	AI produced the complete final EBNF grammar including all design decisions: declarations before statements, fixed student field order, void separated from value types, seven expression precedence layers, for loop with mandatory student keyword and a summary table of every decision with Sebesta references.
Accepted	The overall grammar structure was correct and matched my design. The summary table with Sebesta references was useful for D1.
Rejected / Modified	The <expr> ::= <or_expr> wrapper was unnecessary. I start directly from <or_expr> in my implementation. Also func_call_expr only used IDENT but in my design domain keywords like average() can also be called as expressions.
Errors Caught	AI wrote <expr> ::= <or_expr> as an extra wrapper rule with no purpose. This adds an unnecessary layer to the grammar. I removed it in my final version. AI also missed that domain keywords like average and transcript should be callable inside expressions.
Reflection	<i>Asking AI to produce the final version of something is a good way to catch wrongs. Comparing the output with my own notes revealed two small gaps I had not noticed before.</i>

Entry #5	Date: 2026-05-03 Phase: Lexer / Parser AI Tool: Claude Sonnet 4.6
Goal	Ask AI to implement the complete ScoreScript lexer, parser and AST in Java based on our final EBNF grammar.
Prompt	Now I want to implement ScoreScript in Java. Based on our final EBNF grammar give me all the files: TokenType.java, Token.java, Lexer.java (comments no package), AST node classes, Parser.java, Main.java with --dump-ast support. No package declarations. Parser should reject invalid programs with line numbers in error messages.
Response	AI generated all six Java files. It implemented a recursive descent parser, dedicated token types for student field keywords, FLOAT before INT ordering in the lexer, // comment handling, and --dump-ast flag.
Accepted	The overall architecture — separate files for TokenType, Token, Lexer, AST, Parser, Main. The recursive descent approach matches our grammar structure. FLOAT before INT ordering was correctly implemented. Student field keywords as dedicated token types was a good decision.
Rejected / Modified	The comment syntax used # instead of //. I corrected this. AI also had a package declaration which I removed. The average and rank operations still had wrong syntax in the sample program. AI used average(ClassZ23, math) which is not valid in our grammar.
Errors Caught	AI used # for comments instead of // which we had already decided. It also generated a sample program with average(ClassZ23, math). AI caught and fixed this itself during testing, which shows it does self-correct but only when it actually runs the code.
Reflection	<i>AI is good at generating repetitive code quickly but does not always remember every design decision from the conversation. Testing the output immediately is important. Bugs only appear when you actually run the code.</i>

Entry #6	Date: 2026-05-04 Phase: Testing AI Tool: Claude Sonnet 4.6
Goal	Ask AI to write 3 example ScoreScript programs that together cover all language constructs.
Prompt	I need 3 example ScoreScript programs to test my parser. They should be different from each other and together cover: student declarations, groups, functions with if/else, for loops, curve/average/rank/transcript and variable declarations.
Response	AI generated three programs covering all constructs. During testing a bug appeared. Writing a function call like <code>printLabel(92.0)</code> as a statement caused a parser error. AI fixed it by adding a <code>CallStmt</code> node and checking whether the next token after <code>IDENT</code> is <code>(</code> or <code>=</code> .
Accepted	The three programs and the <code>CallStmt</code> fix. The fix was correct. When the parser sees <code>IDENT</code> followed by <code>(</code> , it should treat it as a function call statement, not an assignment. This is a real design gap I had not noticed.
Rejected / Modified	Nothing major rejected. I removed comments from the source code before putting it in D3 to keep it cleaner.
Errors Caught	The parser could not handle function calls used as statements. AI only found this bug by actually running the code, not by reading the grammar. This shows that testing is always necessary even if everything looks correct on paper.
Reflection	<i>Writing test programs is a good way to find grammar gaps. A bug I did not notice during design showed up immediately when I ran a real program. AI is useful for generating test cases but you still need to verify the output yourself.</i>

Entry #7	Date: 2026-05-04 Phase: Testing AI Tool: Claude Sonnet 4.6
Goal	Test all 5 malformed programs and verify that error messages and line numbers are correct.
Prompt	I tested all 5 error programs and all work correctly. All error messages show the right line numbers.
Response	AI confirmed the milestone and summarized what is completed: EBNF grammar, lexer, parser, AST, --dump-ast flag, 3 valid programs and 5 malformed programs. It also listed what remains for Part 1 deadline.
Accepted	The summary was accurate. All error messages and line numbers matched what I expected.
Rejected / Modified	Nothing rejected. I ran each error program myself in the terminal and verified the output before accepting.
Errors Caught	No errors in this session. However I noticed that error5 initially showed no output. I had not pressed Enter in the terminal. This was my own mistake, not an AI error.
Reflection	<i>Running the programs myself was important. Even though AI said everything worked, I verified each error message and line number personally. Trusting AI output without testing is not enough.</i>

Entry #8	Date: 2026-05-05 Phase: Writing AI Tool: Claude Sonnet 4.6
Goal	Ask AI to generate a D1 document template with §4.2 and §4.3 fully written and placeholders for §4.1 and §4.6.
Prompt	Write D1 Design Specification document for ScoreScript as a Word docx file. Include these sections: 4.1 Language Overview half page, 4.2 Lexical Structure half page, 4.3 Syntax in EBNF 2 pages, 4.6 Names Binding Scope Lifetime 1 page. For 4.1 and 4.6 write placeholder text like [STUDENT WILL WRITE THIS PART] because I should write those parts myself in my own words. Only generate 4.2 and 4.3 fully.
Response	AI generated a complete Word document with cover page, fully written §4.2 and §4.3, and placeholder sections for §4.1 and §4.6 with bullet-point prompts explaining what to write.
Accepted	The §4.2 token tables and §4.3 EBNF were correct and matched our final grammar. The document structure and formatting were professional.
Rejected / Modified	The IDENT pattern was wrong — AI wrote <code>[a-zA-Z_][a-zA-Z0-9_]*</code> with leading underscore allowed. I corrected it to <code>[a-zA-Z][a-zA-Z0-9_]*</code> because we had specifically decided leading underscores are not allowed for readability reasons.
Errors Caught	AI forgot that leading underscores are not allowed in identifiers. This was a deliberate design decision we made earlier in the conversation but AI did not carry it over to the document. I caught it by comparing the document with my own design notes.
Reflection	<i>I wrote §4.1 and §4.6 myself in my own words. AI-generated technical sections like EBNF and token tables saved time but the design rationale sections needed to be written by me. Always check AI output against your own design notes.</i>

Entry #9	Date: 2026-05-07 Phase: Submission AI Tool: Claude Sonnet 4.6
Goal	Verify all Part 1 deliverables are complete before converting to PDF.
Prompt	I finished all the files for Part 1. Before I convert everything to PDF do a final check. I have D1 D3 D4 D5 as docx files. D2 is a zip with src folder examples folder Makefile and README. List what each file should contain for Part 1 and I will confirm everything is in order.
Response	AI read all files and confirmed everything is correct. It listed all deliverables with checkmarks and reminded me about the filename convention with student ID prefix.
Accepted	The checklist was accurate. All deliverables confirmed complete.
Rejected / Modified	Nothing rejected.
Errors Caught	AI reminded me about the filename convention. Files must start with student ID (230104004090_D1_P1.pdf etc.). I might have missed this without the check.
Reflection	<i>Doing a final check before submission is worth the time. AI caught the naming convention detail and confirmed everything was in order.</i>

Part 2 Entries :

Entry #10	Date: 2026-05-17 Phase: Implementation AI Tool: Claude Sonnet 4.6
Goal	Ask AI to implement TypeChecker.java for ScoreScript based on Part 1 files and 8 type rules.
Prompt	These are my Part 1 submitted files for my ScoreScript DSL project. I will paste them so you have the full picture before we start Part 2. Now I need TypeChecker.java. [8 rules + type system rules listed]
Response	AI read all Part 1 files including AST.java, Parser.java, TokenType.java, Token.java, Main.java, README and D1/D3/D4/D5. It identified the exact AST node shapes it needed to walk and generated TypeChecker.java using a 5-pass approach: register all students/groups/function signatures, validate group members, check declaration-section initializers, check top-level statements then check function bodies last against the fully-populated global scope.
Accepted	The overall architecture was correct. The 5-pass approach meant all names were registered before any body was checked, which avoided forward-reference problems. The scope stack (Deque of HashMaps) correctly implements static scoping.
Rejected / Modified	A groupLines field was declared at line 211, mid-class, instead of with the other symbol tables at the top. I asked AI to fix this and it moved the field to the correct location. AI also inlined a single-use helper method lineOfGroup which made the code cleaner and easier to explain.
Errors Caught	AI placed the groupLines field far from the other symbol table declarations. This was a code organization problem that would make the code harder to explain in the exam. I caught it by reading through the file carefully after receiving it.
Reflection	Reading through the generated code carefully before accepting it paid off here. The groupLines field being declared in the middle of the class would have been confusing to explain in an exam if someone asked me to walk through the symbol table design. This taught me that AI can produce working code that still has organizational problems, and that I need to read the output like a reviewer, not just run it and move on.

Entry #11	Date: 2026-05-17 Phase: Implementation AI Tool: Claude Sonnet 4.6
Goal	Ask AI to implement Interpreter.java for ScoreScript.
Prompt	Looks good. Now I need Interpreter.java. [domain operations, control flow, runtime errors, implementation notes listed]
Response	AI first verified TypeChecker compiles cleanly, then generated Interpreter.java. Key design decisions: StudentState inner class keeps mutable grade copies without touching AST nodes, ReturnSignal exception unwinds function bodies on return, scope stack chains function scopes to global scope for static scoping, Integer and Double kept distinct at runtime since type checker guarantees no mixed-mode arithmetic. Programs 1 and 3 ran correctly end-to-end. Program 2 failed the type checker at line 39 — avg >= 90 compared float to int literal, which violated the strict no-coercion rule.
Accepted	The overall architecture was clean. ReturnSignal exception approach for return handling was a good design. StudentState for mutable grades without touching AST was correct. Static scoping implementation matched TypeChecker exactly.
Rejected / Modified	Nothing rejected. I chose Option 1 for the program2.ss issue keep the strict rule and fix the float literals (90 → 90.0) rather than relaxing the type checker.
Errors Caught	AI surfaced the float vs int literal inconsistency in program2.ss by actually running the programs. This was not a bug in the interpreter but a real D1-D2 consistency issue that I had to resolve as a deliberate design decision.
Reflection	The program2.ss issue was actually useful to find at this stage rather than during submission. AI did not silently fix it. It stopped and asked me to make a deliberate decision which forced me to think about what my type system actually says. Choosing Option 1 made the language more consistent and gave me a clearer story to tell in the exam about why no-coercion matters

Entry #12	Date: 2026-05-17 Phase: Implementation AI Tool: Claude Sonnet 4.6
Goal	Update Main.java to wire TypeChecker and Interpreter into the full pipeline.
Prompt	Now I need to update Main.java to wire everything together. Parse as before, run TypeChecker, if it throws print the error and exit, if type check passes run Interpreter, if interpreter throws print the error and exit. Keep the existing --dump-ast flag working. Only update Main.java.
Response	AI updated Main.java with a clean lex → parse → typecheck → interpret pipeline. It added distinct exit codes: 0 success, 1 parse error, 2 usage/file error, 3 type error, 4 runtime error. Every path was verified: normal run, --dump-ast unchanged, type error (exit 3), runtime error via division by zero (exit 4), usage error (exit 2), parse error (exit 1). AI also flagged that empty group [] is a parse error not a runtime error because parseGroupDecl requires at least one member.
Accepted	The exit code design was good . Distinct codes make it easy to distinguish error types in testing and in the Makefile. The --dump-ast preservation was important for backward compatibility with Part 1.
Rejected / Modified	AI removed the old "OK parsed successfully" message since a normal run now executes the program. This was a deliberate and acceptable change. AI also pointed out the README still had the old 0/1/2 exit code table . I noted this as a TODO to fix before submission.
Errors Caught	AI flagged that empty group [] is currently a syntax error, not a runtime error, because the parser requires at least one group member. This means the interpreter's empty-group guards for average/rank cannot be triggered by a literal empty group. AI caught this proactively by actually testing the runtime error path.
Reflection	The exit code table finding was something I would have missed without AI flagging it. Extending exit codes from 0/1/2 to 0/1/2/3/4 is a breaking change for anyone relying on the old scheme and it needed to be documented in the README. This is a good example of AI catching a consistency issue across files that I had not thought about.

Entry #13	Date: 2026-05-17 Phase: Testing AI Tool: Claude Sonnet 4.6
Goal	Resolve the float vs int literal inconsistency in program2.ss and confirm all three programs pass the full pipeline.
Prompt	Option 1. Keep the strict rule, I will fix program2.ss by changing 90 to 90.0, 85 to 85.0, 75 to 75.0, 65 to 65.0, 60 to 60.0 wherever a float is compared to an int literal. Please give me the corrected Interpreter.java and TypeChecker.java files.
Response	AI clarified that neither TypeChecker.java nor Interpreter.java needed any changes. They already implement the strict rule correctly. The fix lived entirely in program2.ss. AI identified exactly five lines to change (39-43) and confirmed that lines 49, 55, 58 were already float-to-float comparisons and needed no change. With the fix applied, all three programs passed the full pipeline cleanly.
Accepted	The analysis was correct. Only program2.ss needed editing, not the implementation files. AI's precise identification of which lines to change and which to leave alone saved time.
Rejected / Modified	Nothing rejected. AI also reminded me to update the D3 report to show the corrected source, which I had not thought about. I made that update.
Errors Caught	AI pointed out that D3 still showed the old <code>avg >= 90</code> source code for program2.ss and that D1 §4.8 should explicitly state the trade-off of the no-coercion decision. Both were consistency issues I had not noticed on my own.
Reflection	This session confirmed that the type checker was working exactly as designed. The float literal issue was not a bug but a consequence of a deliberate design decision. Having to fix program2.ss felt like extra work at first but it actually made the language more consistent and gave me a concrete example to point to when explaining the no-coercion rule in the exam.

Entry #14 — E2	Date: 2026-05-18 Phase: Design Analysis — Experiment E2 AI Tool: Claude Sonnet 4.6
Goal	Ask AI to explain name equivalence vs structural equivalence for the ScoreScript grade map and determine which rule the language uses (Experiment E2).
Prompt	I have a question about my type system for ScoreScript. I have a structured type which is the grades map inside student declarations, it looks like this: grades: { math: 85, science: 90, history: 72 }. Explain the difference between name equivalence and structural equivalence for this type. And does ScoreScript use name equivalence or structural equivalence? I want to understand this in terms of Sebesta 6.14.
Response	AI explained that name equivalence means two types are equivalent only if they share the same named type declaration, while structural equivalence means two types are equivalent if they have the same shape regardless of name. For the grade map specifically, AI explained that the distinction is vacuous by design. The grade map is not a first-class declarable type, so no program can ever put two grade-map types in a position where they need to be compared. ScoreScript uses name equivalence overall, and the type checker only ever compares the four primitive type names (int, float, string, bool).
Accepted	The explanation was correct and matched Sebesta §6.14. The key insight that the structural vs name equivalence question never arises for the grade map because it is non-first-class helped me write §4.5.4 and §4.5.5 more precisely. The exam defense framing AI provided was also useful.
Rejected / Modified	Nothing rejected. I verified the explanation against my D1 §4.5.4 and §4.5.5 to make sure they were consistent, which they were.
Errors Caught	AI raised a consistency warning: if §4.5.4 says name equivalence and §4.5.5 says structural equivalence for the grade map, that would be a contradiction the grader would catch. I checked my D1 and confirmed both sections were already consistent.
Reflection	This experiment made me understand my own type system more precisely. I had written §4.5.4 and §4.5.5 correctly but had not explicitly thought through why the grade map question is vacuous. Now I can answer the exam question confidently: name equivalence applies to the whole language, and for the grade map the question simply never comes up because the type is non-first-class.

Entry #15 — E3	Date: 2026-05-18 Phase: Testing — Experiment E3 AI Tool: Claude Sonnet 4.6
Goal	Run TypeChecker.java on four test programs where I know the expected output and report what happened (Experiment E3).
Prompt	Now I want to do Experiment E3 from the handout. I already asked you to implement TypeChecker.java earlier. Now I need to run it on three test inputs where I know the right answer and report what happened. Write three small ScoreScript programs that test these three specific type errors: 1. A variable declared as int but assigned a float value, 2. A void function result used as a value, 3. A function called with wrong number of arguments.
Response	AI wrote three test programs, each isolating exactly one error with everything else well-typed. All three produced the predicted output: - e3_1.ss → Type error at line 15: cannot initialize int variable 'curveAmount' with a value of type float - e3_2.ss → Type error at line 26: cannot use the result of void function 'printReport' as a value - e3_3.ss → Type error at line 22: function 'letterGrade' expects 1 argument(s) but got 2 AI then suggested adding a fourth negative control program (e3_4.ss) that should pass cleanly, to confirm the checker is not just rejecting everything. I added this and ran it — it passed and executed correctly.
Accepted	All four results matched predictions exactly. The negative control was particularly valuable. It confirmed the checker correctly accepts well-typed code, which the three error tests alone cannot prove.
Rejected / Modified	Nothing rejected. AI's suggestion to add the negative control was correct and I followed it.
Errors Caught	Running e3_4.ss on my own machine revealed a locale bug — the output showed Average: 82,33 instead of 82.33 because Java's printf is locale-sensitive and my Windows machine uses Turkish regional settings. This was fixed with Locale.ROOT in Interpreter.java. The bug was only found because I ran the programs myself rather than relying on AI-run verification.
Reflection	Designing test programs that truly isolate one error is harder than it looks. In test 3, if I had written 90 instead of 90.0 inside letterGrade, the checker would have stopped at the int/float comparison inside the function body and never reached the arity error on line 22. Silently testing the wrong thing. The locale bug finding was also a good reminder that running code on your own machine is essential, not optional.

Entry #16	Date: 2026-05-18 Phase: Testing AI Tool: Claude Sonnet 4.6
Goal	Fix the Java locale bug in Interpreter.java where %.2f printed 82,33 instead of 82.33 on Turkish-locale Windows.
Prompt	Option 1. Fix Interpreter.java to use Locale.ROOT so the output always uses a dot as decimal separator
Response	AI added import java.util.Locale and passed Locale.ROOT to all three printf calls in Interpreter.java — the group-average line, the ranking line, and the transcript-average line. Verified under a simulated tr_TR locale: output now prints 82.33, 91.83, 96.33 with dots. All three type-error tests and program1.ss were unaffected.
Accepted	The fix was minimal and correct — only three printf calls changed, nothing else. Locale.ROOT is the principled choice because it is the locale-neutral root intended for machine-readable output, not a region-specific locale like Locale.US.
Rejected / Modified	Nothing rejected. AI also reminded me to regenerate D3 outputs with the fixed interpreter, which I did.
Errors Caught	AI explained why Locale.ROOT is better than Locale.US. It is the neutral root locale intended for reproducible machine-readable output, not tied to any specific region. This distinction is worth knowing for the exam since the fix is now in my code.
Reflection	This was a small fix but an important one. The bug only appeared on my machine because of Turkish regional settings. Anyone testing on a different machine would have seen correct output and never noticed. This taught me that locale-sensitive formatting is a real portability concern in Java, and that Locale.ROOT is the right tool for any output that needs to be reproducible across environments.

Entry #17	Date: 2026-05-20 Phase: Documentation AI Tool: Claude Sonnet 4.6
Goal	Update README.md exit code table to reflect the Part 2 pipeline changes.
Prompt	The pipeline changed in Part 2 so the README exit code table needs updating. Before it only had three codes because the program just parsed. Now it runs the full pipeline so there are five. Update that section and give me the updated README.
Response	AI updated the exit code table from 3 rows to 5 rows. It also flagged three additional Part 1 leftovers: source tree missing TypeChecker.java and Interpreter.java, example outputs still saying "parsed successfully", and error_type.ss not listed in the invalid programs table. AI asked whether to fix those too.
Accepted	All five fixes in one pass .Source tree, exit codes, example outputs, invalid programs table, and letterGrade float literals. The README is now fully consistent with Part 2.
Rejected / Modified	Nothing rejected.
Errors Caught	AI proactively caught three additional inconsistencies I had not asked about. The letterGrade example still used avg >= 90 which the Part 2 type checker now rejects — I would not have noticed this without AI flagging it.
Reflection	Asking AI to update one thing and having it spot three more problems was useful. The letterGrade example fix was the most important catch — if a grader copied that example and ran it, it would have failed with a type error, which would look bad.

Entry #18	Date: 2026-05-20 Phase: Submission AI Tool: Claude Sonnet 4.6
Goal	Verify all Part 2 deliverables are complete and consistent before converting to PDF.
Prompt	I finished all the files for Part 2. Do a final check on everything before I submit. I will convert to PDF myself, just tell me if anything needs fixing.
Response	AI extracted the D2 zip, compiled from scratch, and ran make test. All eight Java sources compiled cleanly. All three valid programs produced correct output with dot decimals. All five parse error tests and error_type.ss reported correctly. All four E3 programs matched documented messages. D1 validated with all eight sections present and Sebesta references intact. D3 numbers matched verified interpreter runs. D5 had correct grade spread B/A/A/B/A/B. AI flagged one concern: D4 tool labels said Claude Sonnet 4.6 but Part 2 sessions may have used a different model.
Accepted	The full submission was confirmed ready. All deliverables internally consistent.
Rejected / Modified	Nothing rejected. I checked the D4 tool labels and corrected them where needed.
Errors Caught	AI caught the D4 tool label inconsistency — entries said Claude Sonnet 4.6 but the actual Part 2 sessions used a different version. This is an academic integrity concern per handout §9.
Reflection	Doing a final check before submission caught the tool label issue which I had not noticed. This is the same lesson from Part 1 — always verify before submitting, AI can catch details you miss when you are focused on finishing.

AI Conversation Reference

Part 1 interactions (Entries #1–9):

Chat Link: <https://claude.ai/share/7ab7b34e-a816-4d7c-9a4a-4549d57a47a6>

Part 2 interactions (Entries #10–18):

Chat Link: <https://claude.ai/share/2a294c6a-6a2f-4ad6-8ca9-ada63674be16>

AI Tool: Claude Sonnet 4.6 | *Platform:* Claude.ai